

Konspekt Pracy Inżynierskiej

Tytuł:

**Opracowanie narzędzia webowego do wyświetlania i edycji
danych 3D w postaci Gaussian Splatting**

Autor Marcin Kacprzyk

Nr Indeksu 333629

Promotorzy:

dr inż. Wojciech Ostrowski

mgr inż. Łukasz Wilk

Geoinformatyka

Wydział Geodezji i Kartografii

Politechnika Warszawska

Warszawa, 15.06.2024

Spis treści

1 Wprowadzenie

1.1 Motywacja do podjęcia tematu	3
1.2 Cel pracy inżynierskiej	3
1.3 Problem inżynierski	3

2 Technologie

4

3 Harmonogram

4

4 Spis treści

5

5 Postęp pisania pracy

5.1 Utworzenie serwera Vite	6
5.2 Dostosowanie UI Cesium	6
5.3 Konfiguracja aplikacji webowej	7
5.4 Utworzenie podkładowej warstwy siatki	7
5.5 Synchronizacja kamer	8
5.6 Parsowanie pliku PLY	10
5.7 Utworzenie warstwy modelu 3D	12

Literatura

14

Spis rysunków

15

Spis kodów źródłowych

15

1 Wprowadzenie

1.1 Motywacja do podjęcia tematu

Główną motywacją do podjęcia tematu jest potrzeba automatyzacji przetwarzania trójwymiarowych danych przestrzennych. Ważnym impulsem jest pojawienie się technologii Gaussian Splatting, która rewolucjonizuje wizualizację modeli 3D. Nowatorskie podejście zastosowane w tej technologii pozwala na płynne wyświetlanie skomplikowanych modeli 3D w środowisku webowym.

1.2 Cel pracy inżynierskiej

Celem pracy jest opracowanie i implementacja narzędzia webowego umożliwiającego wizualizację, interaktywną eksplorację oraz edycję danych zapisanych w formacie Gaussian Splatting.

1.3 Problem inżynierski

Głównym problemem inżynierskim niniejszej pracy jest przygotowanie platformy webowej pozwalającej na wizualizację i podstawową edycję danych przestrzennych udostępnianych w formie Gaussian Splatting. Wyzwaniem jest połączenie poszczególnych komponentów systemu (wyszukiwarka, mapa podglądowa i viewer 3D, edytor), przy jednoczesnym zachowaniu łatwego w obsłudze interface.

2 Technologie



Implementacja algorytmów



Orientacja przestrzenna obiektów



Interfejs użytkownika

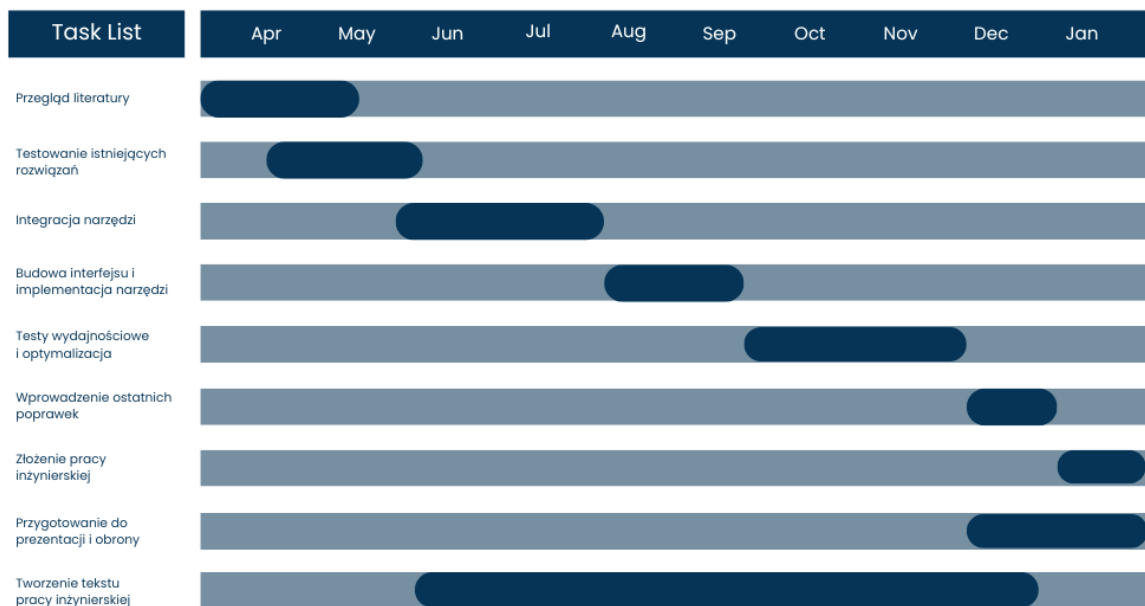


Wyświetlanie zbiorów danych

Rysunek 1: Technologie użyte w pracy inżynierskiej

- **JavaScript** - język programowania służący do implementacji całej logiki aplikacji, parsowania danych binarnych oraz synchronizacji komponentów.
- **Cesium** - biblioteka GIS odpowiedzialna za wyświetlanie trójwymiarowej przestrzeni zarządzanie podkładami mapowymi i orientacji kamery przestrzennej.
- **Deck.gl** – framework wizualizacyjny wykorzystywany do wydajnego renderowania dodatkowych warstw nakładkowych, takich jak siatki współrzędnych czy modele 3D
- **Lit** – biblioteka służąca do stworzenia lekkiego i intuicyjnego interfejsu webowego.

3 Harmonogram



Rysunek 2: Harmonogram realizacji pracy inżynierskiej

4 Spis treści

1. Wprowadzenie
 - 1.1 Cel pracy inżynierskiej
 - 1.2 Problem inżynierski
 - 1.3 Znaczenie i zastosowanie Gaussian Splatting w środowisku webowym
2. Wstęp teoretyczny
 - 2.1 Przegląd istniejących podejść do wizualizacji danych 3D w sieci
 - 2.2 Podstawy matematyczne technologii Gaussian Splatting
 - 2.3 Technologie i standardy renderowania 3D w przeglądarkach webowych
 - 2.4 Przegląd struktur danych i formatów zapisu modeli fotogrametrycznych
3. Opis działania aplikacji
 - 3.1 Strukturacja i parsowanie pliku binarnego PLY
 - 3.2 Ekstrakcja właściwości geometrycznych i optycznych gausów
 - 3.2.1 Współczynniki skali logarytmicznej i kwaterniony rotacji
 - 3.2.2 Kolory bezpośrednie oraz współczynniki pochłaniania
 - 3.3 Transformacje przestrzenne i georeferencja modelu
 - 3.3.1 Przeliczenie lokalnego układu skanera do geograficznego układu współrzędnych ENU
 - 3.3.2 Korekta orientacji przestrzennej (rotacja osi układu pomiarowego do standardu GIS)
 - 3.4 Algorytm synchronizacji kamer
 - 3.5 Generowanie struktur pomocniczych
4. Przedstawienie wyników
 - 4.1 Zbiór danych testowych
 - 4.2 Zestaw parametrów renderowania i konfiguracji wizualnej
 - 4.3 Uzyskane wyniki wizualizacji w środowisku webowym
 - 4.4 Wpływ metod renderowania na jakość wizualną modelu
 - 4.5 Wpływ dynamicznego pozycjonowania celu kamery na komfort nawigacji
5. Analiza wyników
 - 5.1 Ocena jakości i fotorealizmu wyświetlanego modelu
 - 5.2 Analiza błędów geometrycznych i problematycznych przypadków
 - 5.3 Wydajność obliczeniowa aplikacji
 - 5.4 Porównanie z natywnymi, tradycyjnymi metodami wyświetlania chmur punktów
 - 5.5 Wnioski z przeprowadzonej analizy wydajnościowej

6. Implementacja

- 6.1 Architektura narzędzia
- 6.2 Opis najważniejszych modułów
- 6.3 Zewnętrzne biblioteki

7. Dyskusja

- 7.1 Ograniczenia zastosowanej metody
- 7.2 Możliwości dalszego rozwoju narzędzia

8. Podsumowanie

9. Bibliografia

10. Aneksy

- 10.1 Spis rysunków
- 10.2 Spis tabel
- 10.3 Spis kodów źródłowych

5 Postęp pisania pracy

5.1 Utworzenie serwera Vite

Utworzono lokalny serwer Vite, który rozwiązał problem przekazywania pliku PLY do przeglądarki. Podczas prób uruchomienia aplikacji lokalnie przez protokół **file://** występował błąd spowodowany mechanizmami bezpieczeństwa wynikającymi z polityki CORS i Same-Origin Policy, które blokują zapytania o pliki z dysku w celu ochrony danych użytkownika przed nieautoryzowanym dostępem ze strony skryptów.

```
1. import { defineConfig } from 'vite';
2. import cesium from 'vite-plugin-cesium';
3.
4. export default defineConfig({
5.   plugins: [cesium()]
6. });
```

Kod źródłowy 1: Konfiguracja serwera Vite

Serwer uruchamiany jest w konsoli za pomocą polecenia: `npx vite`

5.2 Dostosowanie UI Cesium

Dostosowano domyślny interfejs Cesium poprzez usunięcie zbędnych elementów mogących kolidować z autorskim interfejsem. Usunięto wyszukiwarkę, oś czasu, animacje oraz możliwość wyboru warstw. Ponadto ukryto tekstury globusa, aby nie kolidował wizualnie z modelem.

5.3 Konfiguracja aplikacji webowej

Przygotowano strukturę dokumentu index.html oraz style CSS. Wszystkie warstwy zajmują 100% powierzchni strony przeglądarki i są ułożone warstwowo w odpowiedniej kolejności, przy użyciu parametru z-index, tak aby nie zaburzać widoku warstw Deck.gl. Ponadto wszystkie warstwy Deck.gl zostały „spakowane” do wspólnego kontenera dziedzicząc jego parametry. W kontenerze „deck-container” właściwość pointer-events ustawiono na wartość „none”. Zabieg ten całkowicie uniemożliwia przechwytywanie interakcji myszy przez przezroczystą warstwę wierzchnią, dzięki czemu obrót i przesunięcie kamery są bezpośrednio i płynnie przekazywane do silnika Cesium pod spodem.

```
1. <style>
2. html, body {margin: 0; padding: 0; width: 100%; height: 100%;
   overflow: hidden;}
3.     #appcontainer {
4.         position: relative;
5.         width: 100%;
6.         height: 100%;
7.     }
8.     #cesiumContainer {
9.         position: absolute;
10.        top: 0;
11.        left: 0;
12.        width: 100%;
13.        height: 100%;
14.    }
15.    #deck-container {
16.        position: absolute;
17.        top: 0;
18.        left: 0;
19.        z-index: 10;
20.        width: 100%;
21.        height: 100%;
22.        pointer-events: none;
23.    }
24. </style>
```

Kod źródłowy 2: Konfiguracja stylów warstw

5.4 Utworzenie podkładowej warstwy siatki

Wygenerowano siatkę stanowiącą układ odniesienia dla modelu. Zaimplementowano moduł generujący podkładową siatkę linii o zadanej gęstości. Ponieważ silnik Cesium operuje na globalnych współrzędnych elipsoidalnych, a siatka referencyjna definiowana jest w metrach w lokalnym układzie ENU, konieczne było wykonanie transformacji matematycznej dystansu liniowego na wartości kątowe. Dla uproszczenia obliczeń na lokalnym obszarze testowym przyjęto stałą metryczną długość jednego stopnia szerokości geograficznej (111 132 m). W przypadku długości geograficznej zastosowano skalowanie wartością cosinusa aktualnej szerokości geograficznej środka sceny, co pozwoliło na poprawne odwzorowanie proporcji i odległości w metrach bezpośrednio w układzie współrzędnych aplikacji webowej.

```

1. const createGridLayer = () => {
2.     const gridData = [];
3.     const spacingMeters = 50;
4.     const linesCount = 40;
5.     const half = Math.floor(linesCount / 2);
6.
7.     const latRad = (centerLat * Math.PI) / 180;
8.     const spacingLat = spacingMeters / 111320;
9.     const spacingLon = spacingMeters / (111320 *
    Math.cos(latRad));
10.
11.
12.     for (let i = -half; i <= half; i++) {
13.         const offsetLon = i * spacingLon;
14.         const offsetLat = i * spacingLat;
15.
16.         const maxDistLon = half * spacingLon;
17.         const maxDistLat = half * spacingLat;
18.         gridData.push({
19.             from: [centerLon - maxDistLon, centerLat +
    offsetLat, 0],
20.             to: [centerLon + maxDistLon, centerLat +
    offsetLat, 0]
21.         });
22.         gridData.push({
23.             from: [centerLon + offsetLon, centerLat -
    maxDistLat, 0],
24.             to: [centerLon + offsetLon, centerLat +
    maxDistLat, 0]
25.         });
26.     }
27.     console.log(gridData);
28.     return gridData;
29. };

```

Kod źródłowy 3: Utworzenie podkładowej warstwy siatki

5.5 Synchronizacja kamer

Synchronizacja kamer jest kluczowym elementem architektury aplikacji, pozwalającym na poprawne i jednocześnie wyświetlanie podkładu GIS oraz nakładki z modelem 3D. Ponieważ Cesium i DeckGL to niezależne silniki graficzne brak integracji ich kamer spowodowałoby niezależne poruszanie mapy i modelu. W tym celu wykorzystano zdarzenie pętli renderowania Cesium – `viewer.scene.postRender` – które wywołuje funkcję synchronizującą przy każdym odświeżeniu klatki obrazu. Algorytm pobiera aktualne parametry macierzy widoku z kamery Cesium tj. współrzędne geograficzne punktu, na który patrzy obiekt, wysokość kamery oraz kąty orientacji przestrzennej, które na bieżąco są przekazywane jako nowe wartości widoku warstwy Deck.gl.

```

1. const syncCameras = () => {
2.     const camera = viewer.camera;
3.     const canvas = viewer.scene.canvas;
4.     const centerRay = camera.getPickRay(new
Cesium.Cartesian2(canvas.clientWidth / 2, canvas.clientHeight /
2));
5.
6.     const targetCartesian =
viewer.scene.globe.ellipsoid.radialSelection ? null :
viewer.camera.pickEllipsoid(new
Cesium.Cartesian2(canvas.clientWidth / 2, canvas.clientHeight /
2));
7.     const cartographic =
viewer.scene.globe.ellipsoid.cartesianToCartographic(camera.pos
ition);
8.
9.     if (cartographic) {
10.         const height = cartographic.height;
11.         const zoom = Math.log2(40075016 / (height * 2)) -
1;
12.
13.         let targetLng = centerLon;
14.         let targetLat = centerLat;
15.
16.         if (targetCartesian) {
17.             const targetCarto =
viewer.scene.globe.ellipsoid.cartesianToCartographic(targetCart
esian);
18.             targetLng =
Cesium.Math.toDegrees(targetCarto.longitude);
19.             targetLat =
Cesium.Math.toDegrees(targetCarto.latitude);
20.         }
21.
22.         deckGllayer.setProps({
23.             viewState: {
24.                 longitude: targetLng,
25.                 latitude: targetLat,
26.                 zoom: zoom,
27.                 pitch:
Cesium.Math.toDegrees(camera.pitch) * -1,
28.                 bearing:
Cesium.Math.toDegrees(camera.heading),
29.                 target: [0, 0, 0]
30.             }
31.         });
32.     }
33. };

```

Kod źródłowy 4: Synchronizacja kamer

5.6 Parsowanie pliku PLY

Proces parsowania ma na celu przetworzenie pobranego pliku binarnego PLY do postaci tablic, które mogą zostać bezpośrednio przesłane do pamięci karty graficznej. W celu zoptymalizowania czasu ładowania aplikacji zrezygnowano z zewnętrznych bibliotek parsujących tworząc niskopoziomowy algorytm, realizujący proces w trzech głównych etapach:

1. Detekcja końca nagłówka – każdy plik w formacie PLY typu Binary Little Endian 1.0 posiada nagłówek opisujący parametry każdej elipsoidy. Nagłówek zakończony jest frazą „end_header”, która oddziela go od surowych danych binarnych. Algorytm przeszukuje początkowe bajty pliku jako tekst w poszukiwaniu oczekiwanej frazy.
2. Alokacja bufora typu Float – od miejsca zakończenia nagłówka każdy bajt jest interpretowany jako ciągła tablica liczb zmiennoprzecinkowych.
3. Ekstrakcja atrybutów – znając stałą liczbę atrybutów przypadających na jeden gaus (nagłówek wymienia ich 62), utworzona pętla przechodzi przez bufor za pomocą pętli. W każdej iteracji izolowane są odpowiednie parametry i przypisywane do odpowiednich tablic.

```
1. fetch(ply_file)
2.   .then(res => {
3.     if (!res.ok) {
4.       throw new Error ("Nie można pobrać pliku PLY")
5.     }
6.     return res.arrayBuffer();
7.   })
8.   .then(buffer => {
9.     const bytes = new Uint8Array(buffer);
10.    const headerText = "end_header";
11.    let headerOffset = 0;
12.
13.    for (let i = 0; i < bytes.length - 20; i++) {
14.      const subArray = bytes.slice(i, i +
headerText.length);
15.      const textChunk =
String.fromCharCode.apply(null, Array.from(subArray));
16.      if (textChunk === headerText) {
17.        headerOffset = i + headerText.length;
18.        while (bytes[headerOffset] === 10 ||
bytes[headerOffset] === 13) {
19.          headerOffset++;
20.        }
21.        break;
22.      }
23.    }
24.
25.    const allFloats = new Float32Array(buffer,
headerOffset);
26.    const propertiesPerVertex = 62;
27.    const totalVertices = Math.floor(allFloats.length
/ propertiesPerVertex);
28.
29.    const cleanCoords = new
Float32Array(totalVertices * 3);
30.    const normals = new Float32Array(totalVertices *
3);
31.    const direct_color = new
Float32Array(totalVertices * 3);
```

```

32.     const rest = new Float32Array(totalVertices*45);
33.     const opacity = new Float32Array(totalVertices);
34.     const scale = new Float32Array(totalVertices*3);
35.     const rotation = new Float32Array(totalVertices*
4);
36.     let index_3 = 0;
37.     let restIndex = 0;
38.     let index_4 = 0;
39.     console.log(allFloats, totalVertices);
40.
41.     for (let i = 0; i < totalVertices; i++) {
42.         let baseIndex = i * propertiesPerVertex;
43.         let x = allFloats[baseIndex];
44.         let y = allFloats[baseIndex + 1];
45.         let z = allFloats[baseIndex + 2];
46.         let nx = allFloats[baseIndex + 3];
47.         let ny = allFloats[baseIndex + 4];
48.         let nz = allFloats[baseIndex + 5];
49.         let dc_1 = allFloats[baseIndex + 6];
50.         let dc_2 = allFloats[baseIndex + 7];
51.         let dc_3 = allFloats[baseIndex + 8];
52.         let opacity = allFloats[baseIndex + 53];
53.
54.         scale[index_3] =
55.         Math.exp(allFloats[baseIndex + 54]);
56.         scale[index_3 + 1] =
57.         Math.exp(allFloats[baseIndex + 55]);
58.         scale[index_3 + 2] =
59.         Math.exp(allFloats[baseIndex + 56]);
60.
61.         rotation[index_4] = allFloats[baseIndex +
57];
62.         rotation[index_4 + 1] = allFloats[baseIndex +
58];
63.         rotation[index_4 + 2] = allFloats[baseIndex +
59];
64.         rotation[index_4 + 3] = allFloats[baseIndex +
60];
65.
66.         cleanCoords[index_3] = x;
67.         cleanCoords[index_3 + 1] = -z;
68.         cleanCoords[index_3 + 2] = y;
69.         normals[index_3] = nx;
70.         normals[index_3 + 1] = ny;
71.         normals[index_3 + 2] = nz;
72.         direct_color[index_3] = Math.max(0.0,
73.         Math.min(1.0, dc_1 + 0.5));
74.         direct_color[index_3 + 1] = Math.max(0.0,
75.         Math.min(1.0, dc_2 + 0.5));
76.         direct_color[index_3 + 2] = Math.max(0.0,
77.         Math.min(1.0, dc_3 + 0.5));
78.         for (let j = 0; j < 45; j++) {
79.             rest[restIndex] = allFloats[baseIndex + 9
+ j];
80.             restIndex++;
81.         }
82.         index_3 += 3;
83.         index_4 += 4;
84.     }

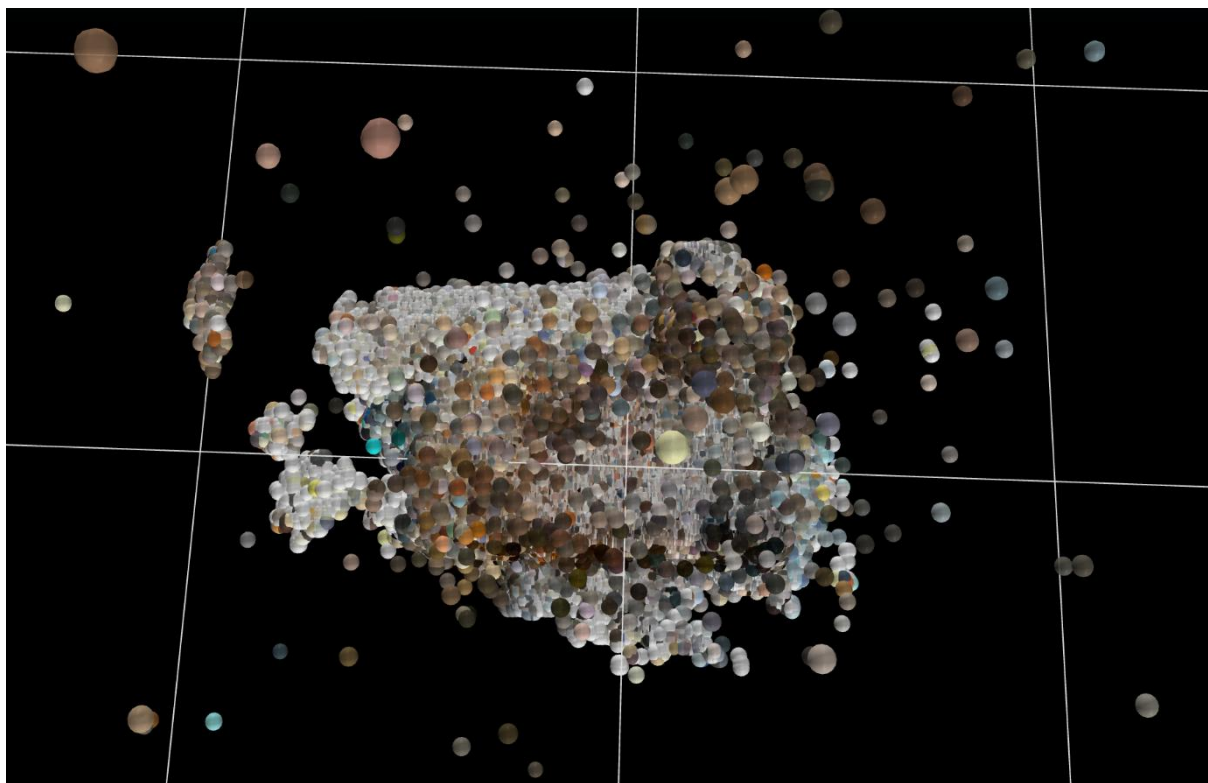
```

Kod źródłowy 5: Parsowanie pliku PLY

5.7 Utworzenie warstwy modelu 3D

Po przeliczeniu współrzędnych modelu 3D wyciągniętych z pliku PLY utworzono kolejną warstwę Deck.gl, do której przypisano odpowiednie parametry modelu i wyświetlono razem z warstwą siatki. Ze względu na swoje ograniczenia Deck.gl nie posiada rodzaju warstwy dedykowanego do wyświetlania danych zapisanych w formacie Gaussian Splatting. Po wielu próbach wyświetlenia modelu 3D nie osiągnięto oczekiwanego efektu, ponieważ każde z rozwiązań posiada braki uniemożliwiające realizację projektu zgodnie z założeniami:

- PLY Loader wbudowany w Deck.gl – przystosowany tylko do tradycyjnych wartości (pozycja XYZ i kolor RGB). Nowoczesne parametry algorytmu Gaussian Splatting były traktowane jako nieznane właściwości i całkowicie ignorowane.
- Biblioteka Luma AI – stworzona na potrzeby e-commerce i gier nie posiada wsparcia dla współrzędnych geograficznych i nie pozwalał na sterowanie kamerą zsynchronizowaną z Cesium. Ponadto Luma AI wymusza konwersję do swojego zamkniętego formatu .luma.
- Domyślne SimpleMeshLayer ze sferami – nieprzystosowana do wyświetlania gaussów, jednak stanowi najbliższą imitację modelu jaką udało się zbudować przy założonych technologiach. Daje efekt „chmury kulek”, które posiadają rzeczywiste kolory modelu i tworzą kształt podobny do oczekiwanego.



Rysunek 3 Wizualizacja modelu 3D i siatki pomocniczej

```

1.  deckGllayer = new Deck({
2.      canvas: 'deck-container',
3.      width: '100%',
4.      height: '100%',
5.      initialState: {
6.          target: [0, 0, 0],
7.          zoom: 15,
8.          pitch: 60,
9.          bearing: 0,
10.     },
11.     controller: false,
12.     parameters: {
13.         depthTest: true,
14.         depthMask: true,
15.     },
16.     layers: [
17.         new LineLayer({
18.             id: 'white-grid',
19.             data: createGridLayer(),
20.             coordinateMode: 0,
21.             coordinateOrigin: [centerLon,
centerLat, 0],
22.             getSourcePosition: d => d.from,
23.             getTargetPosition: d => d.to,
24.             getColor: [255, 255, 255, 220],
25.             getWidth: 2,
26.             widthUnits: 'pixels'
27.         }),
28.         new SimpleMeshLayer({
29.             id: 'ply-gaussian-points',
30.             data: {
31.                 length: totalVertices,
32.                 attributes: {
33.                     getPosition: { value:
lngLatData, size: 3 },
34.                     getColor: { value:
direct_color, size: 3 },
35.                     getSize: { value: size,
size: 3 },
36.                     getOrientation: { value:
rotation, size: 4 },
37.                 }
38.             },
39.             mesh: new SphereGeometry({radius: 1,
nlat: 8, nlong: 8}),
40.             coordinateMode: 1,
41.             coordinateOrigin: [centerLon,
centerLat, 0],
42.             pointSize: 15,
43.             opacity: 0.8,
44.             sizeUnits: 'pixels',
45.         })
46.     ]
47. });

```

Kod źródłowy 6: Wizualizacja modelu 3D i siatki pomocniczej

W ramach testów innych rozwiązań znaleziono technologię PlayCanvas, która pozwala na proste i intuicyjne wyświetlanie formatu Gaussian Splatting w środowisku webowym, natomiast wymaga to zmiany założeń i technologii. Dalsze kroki zostaną podjęte po konsultacji z promotorami.

Literatura

M. Khedr *et al.*, „3D visualization of damaged statues using Gaussian splatting and web interface integration”, *npj Herit. Sci.*, t. 13, nr 1, s. 587, lis. 2025, doi: [10.1038/s40494-025-02063-5](https://doi.org/10.1038/s40494-025-02063-5).

B. Kerbl, G. Kopanas, T. Leimkühler, i G. Drettakis, „3D Gaussian Splatting for Real-Time Radiance Field Rendering”. [Online]. Dostępne na: <https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/>

E. Lopez, „Reading PLY files that are in binary format”, Medium. Dostęp: 28 maja 2026. [Online]. Dostępne na: <https://medium.com/@elopez139/reading-ply-files-that-are-in-binary-format-cab3a37276a2>

„3D Gaussian Splatting as a New Era: A Survey”. Dostęp: 14 kwietnia 2026. [Online]. Dostępne na: <https://arxiv.org/pdf/2402.07181>

N. C. Zakas, *Professional JavaScript for Web Developers*. John Wiley & Sons, 2011.

„Compressed 3D Gaussian Splatting for Accelerated Novel View Synthesis”. Dostęp: 14 kwietnia 2026. [Online]. Dostępne na: https://openaccess.thecvf.com/content/CVPR2024/papers/Niedermayr_Compressed_3D_Gaussian_Splatting_for_Accelerated_Novel_View_Synthesis_CVPR_2024_paper.pdf

„The PLY Format | PlayCanvas Developer Site”. Dostęp: 28 maja 2026. [Online]. Dostępne na: <https://developer.playcanvas.com/user-manual/gaussian-splatting/formats/ply/>

Spis rysunków

1 Technologie użyte w pracy inżynierskiej	3
2 Harmonogram realizacji pracy inżynierskiej	4
3 Wizualizacja modelu 3D i siatki pomocniczej	12

Spis kodów źródłowych

1 Konfiguracja serwera Vite	6
2 Konfiguracja stylów warstw	7
3 Utworzenie podkładowej warstwy siatki	8
4 Synchronizacja kamer	9
5 Parsowanie pliku PLY	10
6 Wizualizacja modelu 3D i siatki pomocniczej	13